July 21, 2009

Builder Pattern - The Swiss Army Knife of Object Creation, Part 2

Are you still eager to learn how we use the Builder Pattern in Java to build hierarchical structures? Like the Groovy People, but without the dynamic Groovyness? In my last article I prepared everything we need for today. Hence take your pen and paper and follow me into the auditorium. The others are already waiting for us...

Let's just sum up, what we've learned last time: Object creation it not always easy. There are classes in the wild that need some special attention while being plugged together. When it comes to the extreme this might blur our focus on the code and the ultimate consequence is: It looses its maintainability. The Builder is proud to offer his helps to us. By encapsulating the actual construction our attention is pushed back to *what* we create, not *how* we create it.

But actually this was just some introduction. Our real goal is finding a declarative way to describe complex structures. Like the Groovy People. But without the Groovyness, only plain old static Java.

Our job for today is declaratively constructing some complex structure. What is a complex structure? I think something hierarchical should be sufficiently complex for our explanations. And the mother of all hierarchical structures is the Document Object Model, of course. Or for the slow people: "We are doing XML, kids!".

Let's create this:

```
<html>
   <head/>
   <body>
       <h1>42 Incorrect Answers</h1>
       Just click this
       <a href="http://42-incorrect-answers.blogspot.com/">link</a>
   </body>
</html>
```

For the sake of education we are doing it the hard way and construct it with `org.w3c.dom`:

```
Element html = doc.createElement("html");
doc.appendChild(html);
Element head = doc.createElement("head");

html.appendChild(head);
Element body = doc.createElement("body");
html.appendChild(body);

Element h1 = doc.createElement("h1");
body.appendChild(h1);
h1.appendChild(doc.createTextNode("42 Incorrect Answers"));
body.appendChild(doc.createTextNode("Just click this "));

Element a = doc.createElement("a");
body.appendChild(a);
a.setAttribute("href","http://42-incorrect-answers.blogspot.com/");
a.appendChild(doc.createTextNode("link"));
```

Yes, it is not the end of the world. But it is a very good example what we've defined above. We surely learn *how* we create XML documents, but have you seen what we create? Although I have written it, I just cannot remember anymore.

On the other hand, what about this?

```
XmlDocumentBuilder builder = new XmlDocumentBuilder();
Document doc = builder
        .beginElement("html")
            .beginElement("head")
            .endElement()
            .beginElement("body")
                .beginElement("h1")
                    .text("42 Incorrect Answers")
                .endElement()
                .text("Just click this ")
                .beginElement("a")
                    .attr("href","http://42-incorrect-answers.blogspot.com/")
                    .text("link")
                .endElement()
            .endElement()
        .endElement()
        .build();
```

Up to now we don't know anything about the XmlDocumentBuilder, but its like a punch in your face: You see exactly what we are doing here! There's one thing I have to admit: We "misused" indention. But that's the secret of declaration: If we declare hierarchical structures, we have to write hierarchical code! Meanwhile you must be quite eager to see how we do this super cool declaration magic. Best would be, we sneak through it, step by step:

```
public class XmlDocumentBuilder {

    private Document document;

    private Stack<Element> stack;

    public XmlDocumentBuilder(DocumentBuilderFactory docBuilderFactory)
            throws ParserConfigurationException {

        this.document = docBuilderFactory.newDocumentBuilder().newDocument();
        this.stack = new Stack<Element>();

    }

    public XmlDocumentBuilder() throws ParserConfigurationException {
        this(DocumentBuilderFactory.newInstance());
    }

    public Document build(){
        return document;
    }
}
```

So far there's nothing magical. We build a Document, therefore we need to remember it. And, as my late grand father used to say: If there is hierarchy, then there's a stack.

```
public XmlDocumentBuilder beginElement(String name){

    Element elem = document.createElement(name);

    // if stack empty, then we have the root element
    if(stack.isEmpty()){
       document.appendChild(elem);
    }
    else {
        stack.peek().appendChild(elem);
    }
```

```
        stack.push(elem);

        return this;
    }

    public XmlDocumentBuilder endElement(){

        if(stack.isEmpty()){
            throw new IllegalStateException("No matching beginElement()");
        }
        stack.pop();

        return this;
    }
```

Yes, this is our working horse: On Groovy they do hierarchy with closures. Since there are none so far in Java, and maybe also not quite soon, there's no way, but we have to find our own method to say something has started and something has ended. We do it by saying something has started and something has ended. (That's easy, does it always work like this?) Starting means: We need a new Element and add it to its parent. Either the current head of the stack, or in the initial case, the document itself. Then we push the Element on the stack and wait for further things to happen. Once everything is done in between, `endElement()` just needs to pop our current element from the stack.

```
        public XmlDocumentBuilder attr(String name,String value){
            if(stack.isEmpty()){
                throw new IllegalStateException("Empty stack");
            }
            stack.peek().setAttribute(name,value);

            return this;
        }

        public XmlDocumentBuilder attr(String name,Object value){
            return attr(name,value.toString());
        }

        public XmlDocumentBuilder text(String text){
            if(stack.isEmpty()){
                throw new IllegalStateException("Empty Stack");
            }
            stack.peek().appendChild(document.createTextNode(text));
            return this;
        }
```

And these are those nodes, which operate on the current stack's head. Either we add attributes or some text nodes. Quite simple.

```
        public XmlDocumentBuilder element(String name){
            beginElement(name);
            return endElement();
        }
    }
```

And finally an example of some convience method: If there's just an element, we don't need this begin/end nonesense, or at the least the Client does not. All the Client wants is just getting his job done. And we help him with it. Sometimes we are so kind...

Basically that's all. We can do Declaration in Java, if we exploit the hierarchy by explicitly demarcating the start and the end of structures. And this is only the Children's Version of it. Consider Immutable Structures: You just have one chance to set everything. And that's in the constructor. Can we do this with a builder as well? Yes we can. We just need a somehow more sophisticate Stack. In this case the Stack's Element type must be Wrapper structure, which could temporarily hold all the values we finally want to put into the Immutable Object. Only in the respective end method, we pop this

intermediate Wrapper from the stack, get its content, push that into constructor of our just right now created, immutable object and finally put it where it belongs.

One question still remains: Where could we use it? As a matter of fact we've been silent about a very important aspect so far: When the Groovy People talk about DSLs (they do this, when they talk about their Builder), they really talk about configuration. Using DSLs is often used in the context of configuring something. If this is a post compile time configuration - if it is supposed to be done by the user and not when the binaries are built - then our whole fancy Builder is nearly worth nothing, since we have to compile it. But on the other hand, if we consider something like the SwingBuilder, where we create static structures, than it's our game.

Furthermore I've already encountered other use cases. For instance consider Unit Tests. Let's take the XmlDocumentBuilder. Just think of some class which does XML binding. It gets an XML document and spills out a model class. Its Unit Test will need XML Documents at some point in time. You could put a real XML file as an artifact into the classpath. But that's just not cool. Because the Unit Test, and the tested data are in two different places (at least 2 different files). Somewhen in future this will deviate. The closer things are together, the less the risk that errors sneak in. Hence if we could easily write the XML Document in code, then we are on the winner side.

Here you can find the sources of the XmlDocumentBuilder and his companion, the XmlDocumentBuilderTest.

This should be enough for now. Feel free to leave your comment and come back in your spare free time. There's always free beer around, and sometimes even some good ideas.